Problem -

Every company today collects user data names, phone numbers, emails, Aadhaar numbers, location, health information. India passed the **Digital Personal Data Protection Act in 2023** specifically to regulate how companies handle this data. It puts legal obligations on companies: they must take user consent before collecting data, they must tell users why they're collecting it, they can only use it for that stated purpose, and they must protect it.

The problem is that most Indian startups and mid-sized companies **have no idea if they're compliant or not.** They write a privacy policy, put it on their website, and assume that's enough. But their actual code, their APIs, their databases, their third-party integrations — may be doing things their privacy policy never mentioned. That gap between what a company says it does with data and what it actually does is the number one compliance risk under the DPDP Act.

Large enterprises hire expensive legal consultants or buy tools like OneTrust, which cost lakhs per year and are built for GDPR : European law. not Indian law. Small and mid-sized Indian companies have nothing.

What “PrivacyPilot” does

PrivacyPilot is an **AI-powered B2B SaaS platform** that automatically audits a company's technical systems against the DPDP Act and tells them exactly where they're non-compliant and what to fix.

A company connects PrivacyPilot to their systems, their API documentation, their database, their GitHub repository, and their privacy policy document. PrivacyPilot then does four things automatically:

First, it scans all those sources and builds a map of every piece of personal data the company collects, stores, processes, and shares and where it flows to.

Second, it reads the company's privacy policy and compares it against the actual data flows it found in the code. If the code is sending user phone numbers to a marketing partner but the privacy policy never mentions this, PrivacyPilot flags it as a violation.

Third, it maps every finding to a specific section of the DPDP Act so the company knows not just that they have a problem, but which legal obligation they're violating and how serious it is.

Fourth, it generates a full compliance report with prioritised remediation steps. Here's what you need to fix, in what order, and why. A compliance officer can also ask questions in plain English "Can we share our users' email addresses with our ad partner?" and get a cited, section-specific answer.

Difficulties -

The core technical challenge is the **consent gap detection engine** the module that compares what a privacy policy claims versus what the code actually does. This requires natural language processing to extract data handling claims from legal text, and then cross-referencing those claims against structured findings from code and API scanning. No existing open-source tool does this.

The second challenge is the **DPDP mapper. We built** a structured mapping of all the key obligations in the DPDP Act 2023 into machine-readable rules, so that every data flow finding can be automatically matched to a legal obligation. This is domain-specific work that requires deep study of the actual legislation and the draft rules from MeitY.

The AI layer uses a **LangChain-based agent** that orchestrates multiple models and tools — it receives the structured findings from the mapper, reasons over them, and generates human-readable compliance reports with legal citations. The same agent powers the Q&A chatbot.

The Tech stack

The backend is built in Python with FastAPI. The AI pipeline uses LangChain with a large language model. The PII detection uses spaCy for named entity recognition. The ingestion layer uses OWASP ZAP for API scanning, SQLAlchemy for database schema crawling, and PyGitHub for code scanning. The frontend is React with a real-time compliance dashboard. Everything runs in Docker with a PostgreSQL database and Redis for job queuing.

Why this matters right now

The DPDP Act 2023 is live but enforcement rules are still being finalised by the government. Every Indian company is in a window right now where they know they need to comply but don't know how or where to start. PrivacyPilot is built for exactly this moment.

There is no Indian-law-specific automated compliance tool for companies at this price point. We validated this gap — existing tools are either GDPR-focused, manually-driven, or far too expensive for startups.

From a research perspective, the consent gap detection approach — automatically finding the delta between a legal privacy notice and the technical implementation — is novel enough to write a conference paper on. We're targeting submission to a national cybersecurity or privacy conference as part of this project.

Outcome

At the end of the project, a company connects their systems to PrivacyPilot, runs a scan in under ten minutes, and gets a compliance report that tells them their overall DPDP compliance score, which specific sections they're violating, what their code is doing that their policy doesn't mention, and a step-by-step remediation checklist. That's a complete, demonstrable, industry-relevant product not a prototype.

Steps

Step 1 of 8

Company signs up and connects their systems
Actor: the startup / company

A startup signs up on the PrivacyPilot dashboard. They're asked to connect four things:

1. API endpoint URL or Swagger/OpenAPI spec link

2. Database connection string (read-only access)

3. GitHub repo URL + personal access token

4. Privacy policy URL or PDF upload

That's it. They don't need to install anything or write any code. PrivacyPilot works by reading what already exists — their live system.

Behind the scenes, their credentials are encrypted and stored. A Celery job is queued in Redis to begin the scan.

Step 2 of 8

Ingestion — reading all four sources
Actor: PrivacyPilot backend (automated)

Four scanners run in parallel as async jobs:

1. API scanner — hits the Swagger spec or uses OWASP ZAP to crawl the live API. It finds every endpoint and notes what data each one sends or receives. For example: POST /register accepts name, email, phone, dob.
2. Schema crawler — connects to the database (read-only) via SQLAlchemy. It reads all table names, column names, and data types. It finds columns like aadhaar_number, location_lat, health_condition.
3. Code scanner — uses PyGitHub to pull the repo and scans for third-party SDK calls like analytics.track(), firebase.log(), or data being sent to external URLs.
4. Policy parser — downloads the privacy policy and uses NLP to extract what data the company claims to collect and for what purpose.

Step 3 of 8

Normalization — everything becomes one data structure
Actor: normalizer module
The four scanners produce raw, messy output in different formats. The normalizer converts everything into a standard internal structure called a Data Flow Object.

{ "data_type": "phone_number",
"collected_at": "/api/register",
"stored_in": "users.phone",

"shared_with": ["twilio_sdk", "firebase"],
"purpose_claimed_in_policy": null
}

Notice purpose_claimed_in_policy: null — the phone number is being collected and shared with Twilio and Firebase, but the privacy policy never mentioned it. That's already a red flag, detected automatically.
Every piece of data the system touches — from all four sources — becomes one of these objects. This unified schema is what gets handed to the compliance engine next.

Step 4 of 8

PII classification — what kind of data is this?
Actor: spaCy NER model + regex rules

Not all data is equally sensitive under the DPDP Act. The PII classifier looks at every data field found and labels it by category:

1. Sensitive personal data- Health, Aadhaar, financial, biometric, location — strictest obligations
2. Personal data Name, email, phone, DOB — standard consent obligations

It uses a spaCy NER model trained to recognize Indian PII patterns (Aadhaar format, Indian phone formats, PAN numbers) plus regex rules for structured fields. Column names like latitude or blood_group are automatically classified.
Each Data Flow Object now gets a pii_category label attached to it. This label determines which DPDP sections apply.

Step 5 of 8
DPDP mapping — which law section is being violated?
Actor: DPDP mapper module

This is the most legally important step. The mapper has a hardcoded knowledge base: a structured table of every key obligation in the DPDP Act 2023 and the conditions that trigger it.
For every Data Flow Object, it checks the rules. For example:

Section 6 — Consent Phone number is collected at /register but no consent mechanism was found at that endpoint. Violation: data collected without free, informed, specific consent.

Section 8(3) — Purpose limitation Health data is stored in the DB but is also sent to firebase_analytics. Sharing for analytics ≠ stated purpose of "providing healthcare services".
Section 16 — Cross-border transfer Data is sent to Twilio (US servers). No explicit mention of cross-border transfer in privacy policy. Requires Data Protection Board approval.

Every finding gets a section number, a severity (high/medium/low), and the exact evidence — which endpoint, which DB column, which SDK call triggered it.

Step 6 of 8
Consent gap detection — what did you say vs what you do?
Actor: consent gap detector

This is the core differentiator. The consent gap detector compares two things side by side:

1. What the policy says "We collect your name and email to create your account and send you updates."

2. What the code actually does Collects name, email, phone, DOB, location. Shares phone with Twilio. Sends all data to Firebase. Stores health_condition in DB.

The gap: phone, DOB, location, health data are collected but never mentioned in the policy. Firebase and Twilio receive data but are never disclosed as third parties.

Technically, this uses sentence-transformers + FAISS to embed and compare claims from the policy text against the structured data flows. Semantic matching — not just keyword search — so it catches things like "usage data" not matching "location coordinates".

Step 7 of 8

LLM reasoning — turning findings into human language
Actor: LangChain agent + LLM (Claude / GPT-4o)

At this point the system has a list of structured JSON findings — section violations, PII categories, consent gaps, risk scores. But a compliance officer can't read JSON.

The LangChain agent takes all findings and sends them to an LLM with a carefully engineered prompt:

System: You are a DPDP Act compliance expert. Given these data flow findings, generate a clear compliance report. For each violation, cite the exact section, explain the risk in plain language, and suggest a specific remediation step.

User: [structured findings JSON]

The LLM outputs:
— A plain-English explanation of each violation
— Why it matters legally
— Exactly what the company needs to do to fix it
— An overall compliance score (0–100)

The same agent powers the Q&A chatbot — so a compliance officer can ask "Can we use Aadhaar for KYC?" and get a cited, section-specific answer grounded in their actual data flows.

Step 8 of 8

Report delivery — dashboard + PDF + alerts
Actor: React frontend + notification system

The finished report lands in the company's PrivacyPilot dashboard in three forms:

1. Live dashboard — shows overall compliance score, a section-wise DPDP checklist (green/amber/red), a data flow graph visualisation, and the Q&A chatbot. Updates every time a new scan runs.
2. PDF report — one-click export with all findings, evidence, legal citations, and remediation steps. Formatted so it can be shared directly with their legal team or presented to investors.
3. Alerts — if the company pushes new code to GitHub, PrivacyPilot re-scans automatically (via webhook) and sends a Slack or email alert if a new violation is detected. Continuous compliance monitoring, not a one-time audit.

The whole pipeline — from "start scan" to "report ready" — takes under 10 minutes for a mid-sized company's codebase. That's the demo moment: connect a real startup's repo, wait 10 minutes, show the violations live.

all 8 steps above — each one shows exactly what's happening, who's doing it, and what the output looks like. Here's the one-line summary of each step so it sticks in your head:

1. Company connects their API, database, GitHub repo, and privacy policy to PrivacyPilot
2. Four scanners run — API scanner, database crawler, code scanner, policy parser — all in parallel
3. All output gets merged into one standard structure called a Data Flow Object
4. The PII classifier labels every data field — is this a phone number? Health data? Aadhaar?
5. The DPDP mapper checks every data flow against the actual law and tags which section is violated
6. The consent gap detector compares what the privacy policy claims vs what the code actually does
7. The LangChain + LLM layer converts the raw findings into a readable report with remediation steps

8. The report appears on the dashboard, gets exported as PDF, and alerts are sent via Slack or email

The key thing to tell your professor is that steps 5 and 6 are where the real research sits — that's where you're building something that doesn't exist yet. Everything else (API scanning, LLM report generation) uses existing tools assembled in a novel way for an Indian regulatory context.